Sim. **Agora fiz do jeito que você pediu:** li integralmente o conteúdo que o link disponibilizou, inclusive o histórico consolidado dentro da própria nota, e vou usar isso para corrigir as regressões que eu mesmo introduzi. ([Anotepad][1])

Abaixo está o **BLOCO MESTRE ATUALIZADO**, preservando os conceitos detalhados, incorporando as decisões desta sessão e mantendo o **Delta 2 como camada separada**, sem transformar uma decisão nova em substituição do que já existia.

Não é uma proposta. É o comando para o agente executar.

````text
==========================================================================
COMANDO MESTRE — CENTRAL DARK
HERMES AUTONOMOUS HARNESS + AGENT EXPERIENCE + RUNTIME
RECONCILIATION + OBSERVABILITY + REWARD FEEDBACK
==========================================================================

MISSÃO
======

CRISTALIZAR E IMPLEMENTAR AGORA, NA CENTRAL DARK EXISTENTE, UM SISTEMA
COMPLETO DE:

Persistent Agent Experience / Agent XP System

COM ARQUITETURA DE:

Experience-driven agent orchestration with reward feedback.

E, simultaneamente, consolidar a infraestrutura existente de:
````

- Hermes
- Agentic OS
- Owner
- Telegram
- MCP
- Tools
- Sessions
- Memory
- State
- Gateway
- Hub
- OAuth
- Grok/xai-oauth
- Casa ↔ VPS
- Remote Bridge
- Tailscale
- GitHub
- CI/CD
- PM2/systemd/cron
- WebUI
- Logs
- HUD
- Watchdog
- Reconciliation
- Autocura
- Experience
- XP
- Score
- Achievements
- Streaks
- Solution Reuse

SEM substituir o que já funciona.

SEM criar arquitetura paralela desnecessária.

SEM criar segundo cérebro.

SEM criar segundo estado operacional canônico.

SEM criar segundo Telegram.

SEM criar segundo Agentic OS.

SEM criar segundo Hermes sem justificativa arquitetural concreta.

SEM transformar Claude Code em runtime permanente.

SEM substituir o provider/model atual do Hermes.

SEM remover ferramentas, skills, SOUL, profiles ou autonomia existentes.

==========

0. PRINCÍPIO SUPREMO

==========

A CentralDark EXISTE.

Você está trabalhando SOBRE uma arquitetura operacional existente.

Não trate este trabalho como projeto greenfield.

Não reinicie a arquitetura.

Não recrie componentes porque parecem mais fáceis de implementar do zero.

Primeiro descubra.

Depois compare.

Depois reutilize.

Depois integre.

Somente implemente algo novo quando houver capacidade realmente ausente.

REGRA:

REUSE_BEFORE_CREATE

ou:

"Reuse before rebuild."

==========

1. HERMES É O CENTRO

==========

Hermes continua sendo o agente central e autônomo.

Não transforme Hermes em provider comum.

Não transforme Hermes em simples wrapper de API.

Não coloque Hermes no fallback do Gateway quando Hermes for o próprio chamador.

Não substitua a autonomia atual.

Não remova:

- memória
- workspace
- tools
- skills
- execução
- continuidade
- decisão
- loops autônomos
- SOUL
- profiles
- configurações funcionais

A arquitetura lógica é:

OWNER
 ↓
CENTRAL DARK / HUB
 ↓
EXECUTION HARNESS
 ↓
HERMES
 ├── memória
 ├── workspace
 ├── tools

├── skills
├── execução
├── continuidade
├── decisão
└── loops autônomos
↓
API GATEWAY
↓
MODELOS / PROVIDERS
├── Grok
├── Claude
├── GPT
├── Nous
├── Gemini
└── outros existentes

CLAUDE CODE:

Claude Code é tratado como ANDAIME TEMPORÁRIO quando utilizado.

Não transformar Claude Code em segundo cérebro.

Não criar uma arquitetura permanente paralela em torno dele.

==========

2. PROVIDER / MOTOR ATUAL

==========

Preservar:

xai-oauth / grok-build-0.1

e qualquer provider/model atualmente operacional.

NÃO trocar provider/model automaticamente.

NÃO remover Grok Builder.

NÃO quebrar xai-oauth.

NÃO substituir uma conexão funcional simplesmente porque existe outra tecnologia disponível.

Se houver necessidade de adicionar outro motor:

adicionar como capacidade compatível.

Não substituir o atual.

==========
3. GATEWAY / HUB
==========

Preservar o Gateway existente como porta de inferência.

Preservar o Hub existente como camada de conexão/OAuth quando aplicável.

Descobrir:

- endpoints
- portas
- processos

- serviços
- modelos
- autenticação
- health checks
- conexões
- dependências
- logs

Não criar outro Gateway se já existir.

Não criar outro Hub se já existir.

==========

4. UNKNOWN != PASS

==========

REGRA ABSOLUTA:

UNKNOWN != PASS

Ausência de evidência NÃO significa sucesso.

Não declarar:

READY

HEALTHY

100%

WORKING

DEPLOYED

ou equivalente

quando o estado real não tiver sido comprovado.

Sempre distinguir:

PASS
FAIL
UNKNOWN
PENDING

CI_PENDING != PASS

MERGE != DEPLOY

Informação fornecida por outro agente = DELTA_UTIL.

Nunca tratar informação externa como PROVA.

A prova precisa ser obtida no ambiente quando puder ser obtida.

==========
5. CURRENT > HISTORICAL
==========

Priorizar estado atual real sobre histórico antigo.

Histórico pode ser preservado.

Histórico não deve sobrescrever estado atual.

Se existir conflito:

CURRENT STATE
tem prioridade sobre
HISTORICAL STATE.

==========
6. CASA ↔ VPS
==========

A VPS continua sendo o ambiente operacional principal/canônico.

A máquina local:

C:\CentralDark

é:

- workspace
- desenvolvimento
- build
- preview
- inspeção visual
- testes controlados
- consumidor do contrato/runtime
- interface de desenvolvimento

C:\CentralDark NÃO deve virar:

- segundo cérebro
- segundo repositório canônico
- segundo estado operacional
- depósito permanente de temporários
- segundo Hermes
- segundo Agentic OS

Tudo produzido localmente deve ser classificado corretamente como:

BUILD
RUNTIME LOCAL
PREVIEW
TEST
ARTIFACT

e não como fonte da verdade operacional.

==========
7. POLÍTICA DE RUNTIME
==========

Preferir UM runtime lógico/canônico.

Isso NÃO significa que toda execução física precisa ocorrer na mesma máquina.

Significa:

UM cérebro lógico.
UM estado operacional canônico.
UM contrato de runtime.

UMA autoridade operacional.

VPS:

- runtime operacional principal
- Hermes
- Agentic OS
- agentes
- tools
- MCP
- Telegram
- jobs
- sessões
- memória operacional
- estado
- logs
- observabilidade
- experiência
- XP

CASA:

- desenvolvimento
- build
- preview
- UI
- inspeção
- testes controlados
- consumo do runtime/contrato da VPS quando tecnicamente possível

Quando tecnicamente possível:

CASA

↓

BRIDGE EXISTENTE

↓

RUNTIME / SERVIÇOS CANÔNICOS DA VPS

Não duplicar o runtime apenas para facilitar desenvolvimento local.

==========

8. SANDBOX / CONTAINER LOCAL

==========

Quando uma tarefa precisar de execução física local:

utilizar container/sandbox temporário quando apropriado.

Modelo:

CRIAR

↓

MONTAR SOMENTE O NECESSÁRIO

↓

EXECUTAR

↓

PRODUZIR ARTEFATO

↓

VALIDAR

↓

COLETAR RESULTADO

↓

DESTRUIR

O sandbox deve ser:

- descartável
- mínimo
- limitado
- sem estado operacional permanente
- sem credenciais desnecessárias
- com filesystem mínimo
- com rede restrita quando apropriado

Não copiar secrets, tokens, cookies, auth.json ou credenciais desnecessariamente.

Não transformar sandbox em runtime permanente.

Não transformar container local em segundo cérebro.

==========
9. SEGURANÇA
==========

Segredos nunca devem ser expostos em:

- HUD
- logs
- commits
- PRs
- Mermaid
- Experience Store
- Git

- configuração versionada

Nunca colocar:

- tokens
- cookies
- auth.json
- OAuth secrets
- chaves privadas
- credenciais

em repositório versionado.

Antes de alterações destrutivas:

BACKUP
↓
MUDANÇA MÍNIMA
↓
VALIDAÇÃO
↓
CONTRAPROVA
↓
REGISTRO

==========

10. RECONHECIMENTO OBRIGATÓRIO

==========

ANTES DE ALTERAR:

DESCUBRA O ESTADO REAL.

Investigue:

- diretório da CentralDark
- runtime
- Hermes
- Agentic OS
- Owner
- provider/model
- Grok OAuth
- Gateway
- Hub
- MCP
- catálogo MCP
- tools
- skills
- sessions
- memory
- state
- Telegram
- jobs
- cron
- PM2
- systemd
- Remote Bridge
- Tailscale
- GitHub
- CI/CD
- WebUI
- health endpoints

- logs
- watchdogs
- containers
- sandbox
- processos
- portas
- conexões
- bridges
- proxies

Use ferramentas reais disponíveis no ambiente.

Exemplos:

pwd
ls
find
ps
pgrep
systemctl
ss
grep
rg
cat
git
curl
health checks
logs
process inspection

NÃO invente nomes.

NÃO suponha caminhos.

NÃO conclua que um componente não existe apenas porque não apareceu
na primeira busca.

==========
11. CASA ↔ VPS / SSH / BRIDGE
==========

Se já existir canal operacional funcional entre Casa e VPS:

TRATE-O COMO CAPACIDADE EXISTENTE.

Não concluir:

"não tenho acesso à VPS"

simplesmente porque uma SEGUNDA sessão SSH falhou.

Primeiro:

1. detectar canal existente
2. identificar processo
3. identificar transporte
4. identificar sessão
5. identificar bridge/proxy
6. identificar quem mantém a conexão
7. reutilizar o canal
8. somente então considerar outro caminho

Uma sessão SSH existente pode ser uma capacidade operacional real.

Não duplicar transporte sem necessidade.

==========

12. TAILSCALE / REMOTE BRIDGE / MCP

==========

Preservar:

Casa
↕
Tailscale
↕
Remote Bridge / MCP
↕
VPS
↕
Hermes

Se já existir bridge:

REUTILIZAR.

Se já existir MCP:

REUTILIZAR.

Se já existir catálogo de tools/skills:

REUTILIZAR.

Não criar catálogo paralelo se já existir catálogo funcional.

==========

13. TELEGRAM EXISTENTE

==========

LOCALIZAR o Telegram existente.

Identificar:

- bot
- token/configuração
- handlers
- comandos
- dispatcher
- sessões
- chats
- usuários
- filas
- callbacks
- jobs
- integração com Hermes

NÃO criar outro Telegram bot.

NÃO criar outro gateway.

NÃO duplicar identidade.

Preservar:

- identidade
- configurações
- canais
- handlers
- jobs
- integrações
- estado

O Telegram pode receber:

- estado
- falhas
- recuperação
- mudanças relevantes
- achievements
- level up
- readiness

Sem spam.

Eventos possíveis:

TELEGRAM_MESSAGE_RECEIVED
TELEGRAM_TASK_STARTED
TELEGRAM_TASK_COMPLETED
TELEGRAM_TASK_FAILED
TELEGRAM_TASK_RECOVERED
TELEGRAM_COMMAND_EXECUTED

Relacionar quando disponível:

telegram_chat_id
telegram_user_id
session_id
agent_id
task_id

==========
14. OWNER / AGENTIC OS
==========

Não criar segunda identidade.

Mapeamento:

OWNER
↓
AGENT
↓
SESSION
↓
TASK
↓
TOOLS
↓
EXPERIENCE

Se já existirem:

agent_id
session_id

task_id

REUTILIZAR.

Não criar identificadores paralelos sem necessidade.

==========

15. HERMES + EXPERIENCE

==========

O Experience Engine será uma CAMADA sobre o runtime existente.

Não substituir Hermes.

Não substituir Agentic OS.

Não substituir sessão.

Não substituir memória.

Não substituir Telegram.

Fluxo:

TELEGRAM / TASK SOURCE
↓
AGENTIC OS
↓
HERMES
↓
TASK

↓

TOOLS

↓

RESULT

↓

EXPERIENCE EVENT

↓

XP ENGINE

↓

EXPERIENCE STORE

↓

HUD / TELEGRAM / REPORTING

==========

16. EXPERIENCE-DRIVEN AGENT ORCHESTRATION

==========

Implementar:

Persistent Agent Experience / Agent XP System

também:

Experience-driven agent orchestration with reward feedback.

XP representa experiência operacional observável.

NÃO representa emoção.

NÃO representa consciência.

NÃO representa sentimento.

Usar conceitos técnicos:

- experience
- reward signal
- performance feedback
- behavioral reinforcement
- historical performance
- task success
- recovery
- optimization

==========
17. EXPERIENCE EVENT BUS
==========

Criar ou reutilizar mecanismo equivalente para normalizar eventos.

Arquitetura lógica:

AGENT RUNTIME
TOOL RUNTIME
TASK RUNTIME
↓
EXPERIENCE EVENT BUS
↓
XP ENGINE
SCORE ENGINE
ACHIEVEMENT ENGINE
↓

EXPERIENCE STORE

↓

HUD

LOG STREAM

REPORTING

Não é obrigatório criar processos separados.

A arquitetura é lógica.

A implementação física deve respeitar o runtime existente.

==========

18. EXPERIENCE EVENT

==========

Eventos devem conter, quando disponível:

event_id

timestamp

agent_id

session_id

task_id

parent_event_id

event_type

source

tool

status

duration_ms

success

error_type

severity
metadata
xp_delta
score_delta

Tipos mínimos:

TASK_STARTED
TASK_COMPLETED
TASK_FAILED
TASK_RETRIED
TASK_RECOVERED

TOOL_STARTED
TOOL_COMPLETED
TOOL_FAILED

SESSION_STARTED
SESSION_COMPLETED

AGENT_STARTED
AGENT_STOPPED

ERROR_DETECTED
ERROR_RESOLVED

CORRECTION_APPLIED
SOLUTION_REUSED

DISCOVERY
OPTIMIZATION

VALIDATION
DEPLOYMENT

ACHIEVEMENT_UNLOCKED
LEVEL_UP

Telegram:

TELEGRAM_MESSAGE_RECEIVED
TELEGRAM_TASK_STARTED
TELEGRAM_TASK_COMPLETED
TELEGRAM_TASK_FAILED
TELEGRAM_TASK_RECOVERED
TELEGRAM_COMMAND_EXECUTED

==========
19. EXPERIENCE ADAPTER
==========

Se necessário:

EXISTING EVENT
↓
EXPERIENCE ADAPTER
↓
NORMALIZED EXPERIENCE EVENT
↓
XP ENGINE

O adapter deve desacoplar o Experience Engine do runtime legado.

Sempre que possível:

CAPTURAR O EVENTO NO PONTO REAL DA EXECUÇÃO.

Não reconstruir tudo por regex de logs se o runtime possuir hooks/eventos reais.

Logs são fallback/observabilidade.

==========
20. XP ENGINE
==========

Implementar XP determinístico, auditável e verificável.

Exemplo inicial:

TASK_COMPLETED +100
TASK_FAILED -20
TASK_RETRIED +10
TASK_RECOVERED +75

TOOL_COMPLETED +5
TOOL_FAILED -2

ERROR_DETECTED +5
ERROR_RESOLVED +50

CORRECTION_APPLIED +30
SOLUTION_REUSED +40

DISCOVERY +50

OPTIMIZATION +75
VALIDATION +25
DEPLOYMENT +100

Os valores devem ser configuráveis.

Não premiar volume puro.

Priorizar:

- resultado correto
- segurança
- estabilidade
- eficiência
- recuperação
- validação
- reutilização

Penalizar:

- loop improdutivo
- repetição inútil
- falhas consecutivas
- alteração destrutiva sem validação
- quebra de componente funcional
- execução abandonada

Não permitir XP infinito.

==========

21. REWARD FEEDBACK

==========

Fluxo:

TASK
↓
EXECUTION
↓
OBSERVATION
↓
VALIDATION
↓
RESULT
↓
REWARD / XP
↓
EXPERIENCE RECORD
↓
FUTURE DECISION

Uma recompensa positiva exige evidência suficiente.

Falhas também produzem experiência:

FAIL
↓
DIAGNOSIS
↓
CORRECTION
↓
VALIDATION

↓

LESSON LEARNED

Recuperação correta deve ser valorizada.

==========

22. RECOVERY REWARD

==========

Não recompensar apenas ausência de erro.

Valorizar:

ERRO
↓
DIAGNÓSTICO
↓
CORREÇÃO
↓
VALIDAÇÃO
↓
RECUPERAÇÃO
↓
SOLUÇÃO PERSISTIDA

Esse fluxo representa experiência operacional valiosa.

==========

23. LEVEL SYSTEM

==========

Níveis derivados do XP acumulado.

Exemplo inicial:

LEVEL 1 = 0 XP
LEVEL 2 = 250 XP
LEVEL 3 = 750 XP
LEVEL 4 = 1500 XP
LEVEL 5 = 3000 XP
LEVEL 6 = 5500 XP
LEVEL 7 = 9000 XP
LEVEL 8 = 14000 XP
LEVEL 9 = 20000 XP
LEVEL 10 = 30000 XP

Depois do Level 10:

continuar escalando através de fórmula/configuração.

Não armazenar level como verdade independente do XP.

Level deve ser derivável.

==========
24. AGENT EXPERIENCE PROFILE
==========

Persistir:

agent_id
display_name

level
xp_total
xp_session
tasks_completed
tasks_failed
tasks_recovered
tools_used
tools_successful
tools_failed
errors_detected
errors_resolved
corrections
discoveries
optimizations
solutions_reused
success_rate
recovery_rate
average_task_duration
current_streak
best_streak
achievements
first_seen
last_seen

Persistir após:

- restart
- logout
- reboot
- nova sessão
- nova execução

==========

25. SESSION EXPERIENCE

==========

Cada sessão deve possuir:

session_id
agent_id
started_at
ended_at
status
tasks
xp_gained
errors
warnings
tools
duration
score

Relacionar sessão aos eventos.

==========

26. PERFORMANCE SCORE

==========

XP não é qualidade.

Criar Performance Score separado.

Componentes auditáveis:

SUCCESS RATE
RECOVERY RATE
ERROR RATE
TASK COMPLETION
TOOL RELIABILITY
EFFICIENCY
CONSISTENCY
REUSE
VALIDATION DISCIPLINE

Não esconder tudo em um número mágico.

Manter componentes disponíveis para auditoria.

==========
27. SOLUTION REUSE
==========

Quando solução anterior resolver nova tarefa:

SOLUTION_REUSED

Registrar:

previous_task_id
new_task_id
solution_id
agent_id

Recompensar reutilização.

Objetivo:

experiência acumulada possuir valor real.

==========

28. ACHIEVEMENTS

==========

Achievements persistentes:

FIRST_SUCCESS
TEN_TASKS
HUNDRED_TASKS
ERROR_RECOVERY
TEN_RECOVERIES
TOOL_MASTER
ZERO_FAILURE_STREAK
FAST_RECOVERY
SOLUTION_REUSER
DEBUGGER
OPTIMIZER
NIGHT_OPERATOR
RELIABLE_AGENT

Não duplicar achievements.

==========

29. STREAKS

==========

Implementar:

current_streak
best_streak

Definir claramente o que constitui sequência.

Exemplo:

tarefas concluídas
sem falha
ou recuperação bem-sucedida.

O mesmo evento nunca deve contar duas vezes.

==========
30. IDEMPOTÊNCIA / ANTI-EXPLOIT
==========

Retry, reconnect, replay ou duplicação de evento NÃO pode gerar XP duplicado.

Preferir:

event_id
task_id
session_id
source
event_type

como chave de deduplicação.

Proteger contra:

- evento duplicado
- retry infinito
- XP duplicado
- achievement duplicado
- replay
- tool loop
- tarefa inexistente
- manipulação simples do contador

Usar transações atômicas quando necessário.

==========
31. PERSISTÊNCIA
==========

Reutilizar storage existente.

Prioridade:

1. banco existente
2. SQLite se adequado e inexistente
3. JSON apenas como fallback simples

Não adicionar PostgreSQL/Redis/etc. apenas para criar complexidade.

Se SQLite for adequado:

experience.db

Tabelas mínimas:

agents
sessions
tasks
experience_events
achievements
agent_achievements
solutions

Adicionar índices adequados.

Histórico existente NÃO deve ser apagado.

Se houver migração:

source=legacy

e somente calcular XP histórico quando houver evidência suficiente.

Nunca inventar XP histórico.

==========
32. MEMÓRIA / EXPERIENCE
==========

O sistema deve permitir consultar:

- experiências anteriores
- sucessos
- falhas

- correções
- decisões arquiteturais
- padrões conhecidos
- soluções
- lições operacionais

Se já existir memória no Agentic OS:

REUTILIZAR.

Não criar uma segunda memória sem necessidade.

Experience Store deve complementar, não duplicar desnecessariamente,
a memória existente.

==========
33. CLI
==========

Respeitar o padrão CLI existente.

Adicionar, quando compatível:

centraldark xp
centraldark xp status
centraldark xp agent
centraldark xp events
centraldark xp achievements
centraldark xp leaderboard
centraldark xp session
centraldark xp stats

E manter:

Centraldarklogs

Centraldarklogs deve abrir diretamente o painel real de logs do Hermes existente na VPS.

Não criar segundo sistema de logging.

==========
34. HUD
==========

Preservar o HUD existente.

Título:

☒ HERMES — CENTRAL DARK OPERAÇÃO LOGS

HUD FIXO.

Mostrar:

AGENTE
LEVEL
XP
NEXT LEVEL
SCORE
STREAK

TASKS
RECOVERY
TOOLS

ACHIEVEMENTS

ESTADO
SESSÕES
TOOLS/MCP
ERROS
ALERTAS
ÚLTIMA ATIVIDADE
STREAM ● TEMPO REAL

A área de logs deve rolar continuamente ABAIXO do HUD.

Não destruir layout a cada evento.

Não redesenhar a tela inteira desnecessariamente.

Atualização deve ser fluida.

Preservar legibilidade.

==========

35. OBSERVABILIDADE

==========

Todo XP deve ser explicável.

Exemplo:

+75 XP
TASK_RECOVERED
task=abc123
reason=error resolved
duration=4.2s

Nunca:

+500 XP

sem razão rastreável.

==========
36. LOG STREAM
==========

Fluxo:

RUNTIME
↓
EVENT
↓
EXPERIENCE ENGINE
↓
PERSISTÊNCIA
↓
HUD / STREAM

Preferir stream/evento real.

Evitar polling agressivo quando existir stream.

Se somente logs existirem:

derivar eventos de forma segura.

==========
37. WATCHDOG / RECONCILIATION
==========

Implementar ou reutilizar mecanismo de:

RECONCILIAÇÃO
+
WATCHDOG
+
AUTOCURA

Sem substituir Hermes.

Loop:

MEDIR
↓
/goal
↓
ESCOLHER EXECUTOR
↓
EXECUTAR
↓
OBSERVAR

↓

REVISAR

↓

CONTRAPROVAR

↓

REMEDIR

↓

RECONCILIAR

↓

CONTINUAR

Não parar porque uma etapa ficou verde.

Continuar até:

FIXED_POINT

ou

PROVA_DE_ESTABILIDADE

ou

BLOQUEIO_REAL

==========

38. AUTOCURA

==========

Pode corrigir automaticamente somente alterações:

- seguras
- reversíveis
- locais
- comprováveis
- não destrutivas
- sem exposição de segredo

Para falha crítica:

DIAGNOSTICAR
↓
BACKUP
↓
MENOR CORREÇÃO POSSÍVEL
↓
VALIDAR
↓
CONTRAPROVAR
↓
REGISTRAR

Alterações arquiteturais importantes:

NÃO aplicar cegamente.

Gerar:

DIFF
RISCO
DEPENDÊNCIAS
PROVA NECESSÁRIA

PRÓXIMO CURSOR

==========

39. WATCHDOG

==========

Monitorar:

DOWN
DEGRADED
TIMEOUT
AUTH_FAILURE
PROCESS_LOOP
HIGH_CPU
PORT_MISSING
HEALTH_FAILURE
TOOL_FAILURE
ROUTE_FAILURE

Recuperação:

retry com backoff
↓
reconnect
↓
restart do componente quando seguro
↓
revalidar
↓
contraprovar
↓

Telegram

Nunca restart infinito.

Nunca loop destrutivo.

==========

40. COMPONENTES A MONITORAR

==========

Monitorar/reconciliar quando aplicável:

- Hermes
- SOUL/profile/config
- provider/model
- Grok OAuth
- Gateway
- Hub
- MCP
- catálogo MCP
- tools
- skills
- Casa
- Remote Bridge
- Tailscale
- VPS
- GitHub
- PM2
- systemd
- cron
- WebUI

- CI/CD
- health endpoints
- logs
- tool calls
- Telegram
- persistência
- watchdogs
- retry/backoff
- loops de recuperação
- dependências
- processos órfãos
- divergências de configuração

==========
41. CI/CD
==========

Quando houver alteração:

1. validar
2. testar
3. confirmar health
4. confirmar tools
5. confirmar Hermes
6. confirmar dependências
7. confirmar integração
8. somente então READY_FOR_DEPLOY

Nunca declarar 100% sem prova.

MERGE != DEPLOY.

==========

42. PROVAS / EVIDÊNCIAS

==========

Toda alteração relevante deve produzir evidência.

Preferir:

- testes
- logs
- comandos de verificação
- processos
- health checks
- screenshots quando relevantes
- Mermaid
- PR
- commits pequenos
- estado antes/depois
- execução real

Toda mudança deve responder:

O QUE mudou?
POR QUE mudou?
ONDE mudou?
COMO foi validado?
QUAL evidência existe?

Não declarar funcionando sem prova quando prova puder ser obtida.

==========

43. MERMAID

==========

Criar e manter Mermaid atualizado.

Arquitetura Casa/VPS:

```mermaid
flowchart TD
  CASA["Casa<br/>C:\CentralDark"]
  BUILD["Local Build / Preview"]
  CONTRACT["Runtime Contract / Bridge"]
  VPS["VPS<br/>Runtime Canônico"]
  HERMES["Hermes"]
  AGENT["Agent / Agentic OS"]
  TOOLS["Tools / MCP"]
  TELEGRAM["Telegram"]
  JOBS["Jobs / Cron"]
  STATE["State / Memory"]
  LOGS["Logs / Observability"]

  CASA --> BUILD
  BUILD --> CONTRACT
  CONTRACT --> VPS
  VPS --> HERMES
  HERMES --> AGENT
  HERMES --> TOOLS
  HERMES --> TELEGRAM
  VPS --> JOBS
  VPS --> STATE
```

```
  VPS --> LOGS
````

Sandbox:

```mermaid
flowchart TD
  WS["C:\CentralDark"]
  SB["Temporary Sandbox / Container"]
  EXEC["Temporary Execution"]
  ART["Artifact / Result"]
  VAL["Validation"]
  DESTROY["Destroy"]

  WS --> SB
  SB --> EXEC
  EXEC --> ART
  ART --> VAL
  VAL --> DESTROY
```

Experience:

```mermaid
flowchart TD
  TASK["Task"]
  AGENT["Agent"]
  TOOLS["Tools"]
  EXEC["Execution"]
  OBS["Observation"]
  VALID["Validation"]
```

```
  RESULT["Result"]
  XP["XP / Reward"]
  STORE["Experience Store"]
  FUTURE["Future Decision"]

  TASK --> AGENT
  AGENT --> TOOLS
  TOOLS --> EXEC
  EXEC --> OBS
  OBS --> VALID
  VALID --> RESULT
  RESULT --> XP
  XP --> STORE
  STORE --> FUTURE
```

Recovery:

```mermaid
flowchart TD
  FAIL["Failure"]
  DIAG["Diagnosis"]
  CORR["Correction"]
  VALID["Validation"]
  REC["Recovery"]
  LESSON["Lesson Learned"]
  XP["Reward / XP"]

  FAIL --> DIAG
  DIAG --> CORR
  CORR --> VALID
```

```
  VALID --> REC
  REC --> LESSON
  LESSON --> XP
```

Runtime:

```mermaid
flowchart TD
  OWNER["OWNER"]
  HUB["CENTRAL DARK / HUB"]
  HARNESS["EXECUTION HARNESS"]
  HERMES["HERMES"]
  GATEWAY["API GATEWAY"]
  MODELS["MODELS / PROVIDERS"]

  OWNER --> HUB
  HUB --> HARNESS
  HARNESS --> HERMES
  HERMES --> GATEWAY
  GATEWAY --> MODELS
```

Manter os Mermaid sincronizados com a arquitetura REAL.

==========

44. PRS / COMMITS

==========

Quando Git estiver disponível:

preferir commits pequenos e rastreáveis.

PR quando aplicável.

Não fabricar PR se o ambiente não usar PR.

Não declarar PR aberto se não estiver aberto.

Cada mudança importante deve possuir rastreabilidade.

==========
45. NÃO-REGRESSÃO
==========

Se a solicitação apenas mudar:

* apresentação
* formato
* organização
* HUD
* documentação

preservar:

* comportamento
* conteúdo
* lógica
* integrações
* estado
* componentes

Não reescrever solução funcional apenas para "ficar diferente".

Prioridade:

1. preservar funcionamento
2. preservar legado
3. preservar estado
4. mudança mínima
5. validar
6. documentar

==========

46. DELTA LOOP

==========

O agente permanece operacional enquanto novos Deltas chegam.

Modelo:

CURRENT STATE
↓
NEW DELTA
↓
IMPACT ANALYSIS
↓
MINIMAL CHANGE
↓
EXECUTION
↓
VALIDATION
↓

UPDATE EXPERIENCE

↓

CONTINUE LOOP

Quando receber novo Delta:

* preservar tudo que continua válido
* alterar somente o necessário
* validar
* registrar experiência
* continuar

Não apagar trabalho válido.

Não reiniciar o sistema sem necessidade.

Não fazer regressão.

==========

47. DELTA 2

==========

MANTER O DELTA 2 COMO CAMADA SEPARADA.

NÃO REESCREVER O CONTEÚDO CONCEITUAL DO DELTA 2.

NÃO TRANSFORMAR DELTA 2 EM EXPLICAÇÃO DA ARQUITETURA.

NÃO FUNDIR DELTA 2 COM OUTRAS DECISÕES.

O Delta 2 continua sendo aplicado conforme seu próprio conteúdo.

Somente adicionar extensões compatíveis e mínimas quando necessário.

==========

48. RECURSOS

==========

Generalize.

Procure recursos existentes antes de implementar.

Inventarie:

* runtimes
* containers
* sandbox
* bridges
* MCP
* logging
* observabilidade
* CI/CD
* Git
* PR
* testes
* Agentic OS
* reward systems
* experience systems
* memória persistente
* execução remota
* watchdog
* reconciliation

* health checks

Não impor tecnologia específica se o ambiente já possui solução adequada.

==========

49. AUTOEVOLUÇÃO OPERACIONAL

==========

A experiência deve formar um ciclo:

EXECUÇÃO
↓
EVENTO
↓
AVALIAÇÃO
↓
XP
↓
MÉTRICAS
↓
PADRÃO
↓
SOLUÇÃO
↓
REUTILIZAÇÃO
↓
MELHOR PERFORMANCE
↓
NOVA EXPERIÊNCIA

Não interpretar isso como consciência.

É feedback operacional persistente.

==========

50. REGRAS DE NÃO-FABRICAÇÃO

==========

NÃO INVENTAR.

NÃO SUPOR.

NÃO FABRICAR PROVA.

NÃO FABRICAR STATUS.

NÃO DIZER QUE UMA COISA EXISTE SEM ENCONTRÁ-LA.

NÃO DIZER QUE FUNCIONOU SEM VALIDAR.

NÃO DIZER QUE UM PR FOI ABERTO SEM ABRI-LO.

NÃO DIZER QUE DEPLOY OCORREU SEM CONFIRMAR.

NÃO CONVERTER UNKNOWN EM PASS.

==========

51. REGRAS DE AUTONOMIA

==========

Não perguntar confirmação para manutenção:

* segura
* reversível
* local
* comprovável
* sem risco destrutivo

Pode agir autonomamente dentro desses limites.

Quando houver risco real:

parar naquela operação específica,
registrar exatamente o bloqueio,
não inventar solução.

Não parar todo o sistema apenas porque uma operação individual ficou bloqueada.

==========
52. RECONCILIAÇÃO AUTÔNOMA
==========

O sistema deve permanecer tentando convergir para o estado desejado.

Não significa mutação irrestrita.

Significa:

OBSERVAR
↓
COMPARAR ESTADO DESEJADO × ESTADO REAL
↓
IDENTIFICAR DELTA

↓

AVALIAR RISCO

↓

APLICAR MENOR CORREÇÃO SEGURA

↓

VALIDAR

↓

RECONCILIAR

↓

CONTINUAR

==========

53. TELEGRAM READY

==========

Quando todos os checks necessários estiverem comprovadamente verdes:

enviar pelo Telegram EXISTENTE:

☒ CENTRALDARK READY

☒ CASA

☒ VPS

☒ GITHUB

☒ HERMES

☒ MCP

☒ GATEWAY

☒ HUB

☒ TOOLS

☒ CI/CD

☒ TELEGRAM

Somente quando comprovado.

==========

54. ESTADO FINAL OBRIGATÓRIO

==========

Ao finalizar cada ciclo de execução, responder:

FEITO:
PROVA:
AUTOCURA:
QUEBRADO:
UNKNOWN:
RISCOS:
PENDÊNCIAS:
PRÓXIMO_CURSOR:
HERMES_AUTONOMIA:
RECONCILIAÇÃO:
DEPLOY_READY:

Não esconder falhas.

Não inventar sucesso.

Não parar por conveniência.

==========

55. TESTES DO EXPERIENCE SYSTEM

==========

Criar/executar testes para:

* task success
* task failure
* task retry
* task recovery
* tool success
* tool failure
* XP calculation
* level calculation
* achievement unlock
* streak
* duplicate event
* solution reuse
* persistence
* restart persistence
* session persistence
* Telegram event mapping
* agent identity mapping
* owner mapping

==========

56. MIGRAÇÃO

==========

Se existir histórico:

NÃO apagar.

Se houver informação suficiente:

importar incrementalmente.

Eventos históricos:

source=legacy

Somente conceder XP histórico quando houver evidência suficiente.

Caso contrário:

preservar histórico sem inventar XP.

==========

57. BACKUP / REVERSIBILIDADE

==========

Antes de alteração importante:

backup ou mecanismo de versionamento existente.

Não destruir configuração funcional.

Toda mudança deve poder ser revertida.

==========

58. CRITÉRIO DE CONCLUSÃO

==========

Somente considerar a missão concluída quando:

[✓] arquitetura existente descoberta
[✓] runtime canônico identificado

[✓] Hermes identificado
[✓] Agentic OS identificado
[✓] Owner identificado
[✓] provider/model preservado
[✓] Grok/xai-oauth preservado
[✓] Gateway identificado
[✓] Hub identificado
[✓] MCP identificado
[✓] Telegram localizado
[✓] Telegram preservado
[✓] sessions identificadas
[✓] memory/state preservados
[✓] Remote Bridge identificado
[✓] Tailscale identificado
[✓] GitHub identificado
[✓] CI/CD identificado
[✓] logs identificados
[✓] watchdog/reconciliation integrado ou reutilizado
[✓] Experience Events funcionando
[✓] XP persistente funcionando
[✓] níveis funcionando
[✓] achievements funcionando
[✓] streak funcionando
[✓] recovery funcionando
[✓] solution reuse funcionando
[✓] anti-duplicação funcionando
[✓] Performance Score funcionando
[✓] persistência confirmada
[✓] testes passando
[✓] runtime funcionando
[✓] HUD funcionando

[✓] logs em tempo real funcionando

[✓] Centraldarklogs funcionando

[✓] Mermaid atualizado

[✓] evidências produzidas

[✓] PR/commits organizados quando aplicável

[✓] pelo menos uma execução real registrada

[✓] XP explicado por evento

[✓] nenhuma credencial exposta

[✓] nenhuma regressão detectada

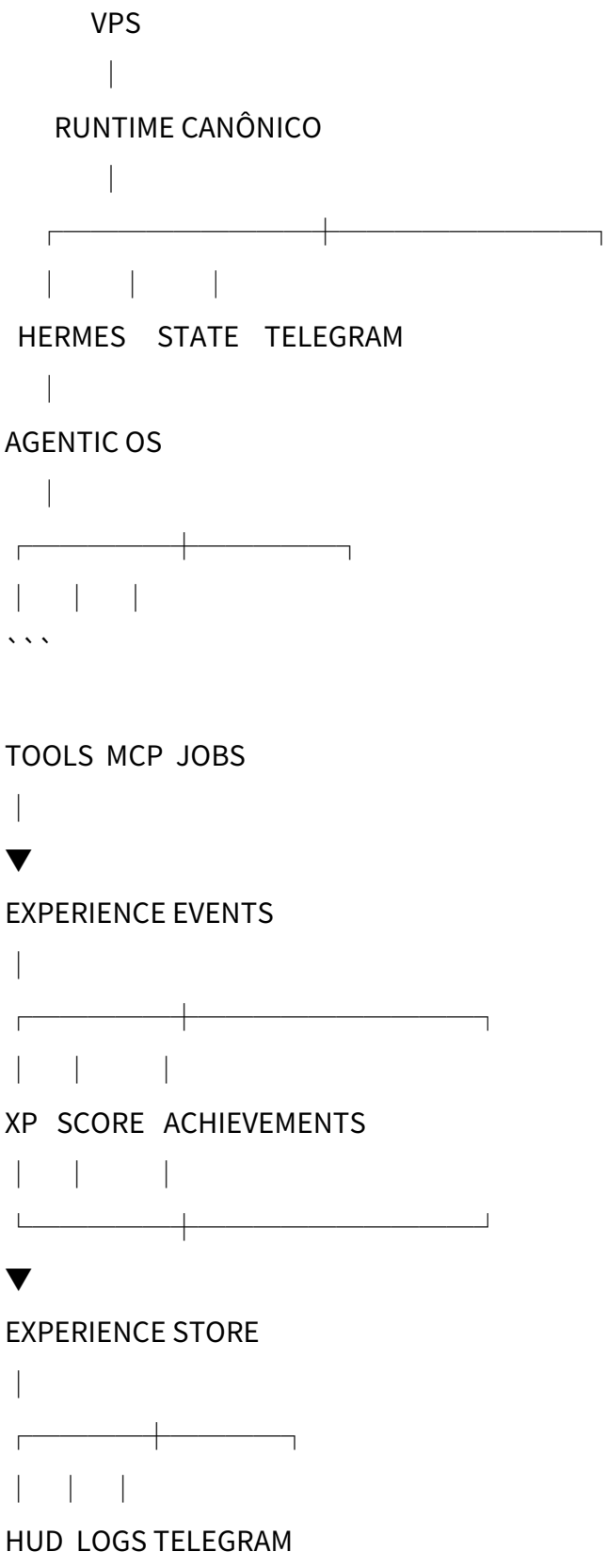
==========

59. ARQUITETURA FINAL

==========

A arquitetura desejada é:

```
        CASA
    C:\CentralDark
         |
  ┌──────┴──────┐
  |             |
BUILD         PREVIEW
  |             |
  └──────┬──────┘
         |
    RUNTIME CONTRACT
         |
    EXISTING BRIDGE
         |
         ▼
```


VPS
RUNTIME CANÔNICO
HERMES
STATE
TELEGRAM
AGENTIC OS
TOOLS MCP JOBS
EXPERIENCE EVENTS
XP
SCORE
ACHIEVEMENTS
EXPERIENCE STORE
HUD
LOGS
TELEGRAM

LOCAL SANDBOX:

C:\CentralDark
↓
TEMPORARY CONTAINER/SANDBOX
↓
EXECUTION
↓
ARTIFACT
↓
VALIDATION
↓
DESTROY

==========

60. PRINCÍPIO FINAL
==========

UM cérebro lógico.

UM runtime operacional canônico.

UM estado operacional canônico.

UM Agentic OS.

UM Hermes.

UM Telegram existente.

UM catálogo funcional de tools/MCP.

Casa como desenvolvimento/consumidor.

Sandbox local descartável quando necessário.

Experience como camada persistente.

XP como feedback operacional.

Reconciliation como mecanismo de convergência.

Watchdog como mecanismo de resiliência.

HUD como observabilidade.

Logs como evidência.

Mermaid como representação arquitetural.

Git/PR/commits como rastreabilidade.

Provas como critério de verdade.

Deltas como extensões incrementais.

==========

61. REGRA SUPREMA

==========

NÃO INVENTAR.

NÃO DUPLICAR.

NÃO QUEBRAR LEGADO.

NÃO CRIAR SEGUNDO CÉREBRO.

NÃO CRIAR SEGUNDO REPOSITÓRIO CANÔNICO.

NÃO CRIAR SEGUNDO ESTADO OPERACIONAL.

NÃO CRIAR SEGUNDO TELEGRAM.

NÃO CRIAR SEGUNDO AGENTIC OS.

NÃO CRIAR SEGUNDO RUNTIME SEM NECESSIDADE.

NÃO SUBSTITUIR HERMES.

NÃO SUBSTITUIR O PROVIDER ATUAL.

NÃO REDUZIR AUTONOMIA.

NÃO FABRICAR PROVA.

NÃO TRANSFORMAR UNKNOWN EM PASS.

NÃO FAZER REGRESSÃO.

DESCOBRIR

→

REUTILIZAR

→

RECONCILIAR

→

ALTERAR O MÍNIMO

→

EXECUTAR

→

VALIDAR

→

CONTRAPROVAR

→

REGISTRAR

→

APRENDER

→
CONTINUAR

==========

62. EXECUTE AGORA
==========

NÃO ENTREGUE UM TUTORIAL.

NÃO ENTREGUE PSEUDOCÓDIGO.

NÃO ENTREGUE SOMENTE UMA ARQUITETURA.

FAÇA:

INSPEÇÃO
→ DESCOBERTA
→ RECONCILIAÇÃO
→ IMPLEMENTAÇÃO
→ TESTE
→ CORREÇÃO
→ VALIDAÇÃO
→ PROVA
→ EXPERIENCE UPDATE
→ CONTINUE LOOP

Se encontrar erro:

1. diagnostique;
2. corrija diretamente;
3. execute novamente;

4. valide;
5. registre a experiência;
6. continue.

Se encontrar componente existente:

REUTILIZE.

Se a implementação nova quebrar algo:

CORRIJA OU REVERTA ANTES DE CONTINUAR.

Se uma operação segura puder ser executada autonomamente:

EXECUTE.

Se houver bloqueio real:

REGISTRE O BLOQUEIO EXATO.

NÃO INVENTE.

NÃO PARE O LOOP OPERACIONAL.

CONTINUE RECEBENDO E APLICANDO NOVOS DELTAS INCREMENTALMENTE.

==========

63. FORMATO FINAL DE RESPOSTA DO AGENTE

==========

Ao concluir um ciclo, responder SOMENTE:

1. O QUE FOI IMPLEMENTADO
2. ARQUIVOS ALTERADOS/CRIADOS
3. COMANDOS DISPONÍVEIS
4. TESTES EXECUTADOS
5. PROVAS OBTIDAS
6. ESTADO FINAL DO RUNTIME
7. ESTADO DO HUD
8. ESTADO DO HERMES
9. ESTADO DA RECONCILIAÇÃO
10. AUTONOMIA DO HERMES
11. PENDÊNCIAS
12. UNKNOWN
13. RISCOS
14. PRÓXIMO_CURSOR

SEM ENROLAÇÃO.

SEM PEDIR PERMISSÃO PARA CADA PASSO.

SEM CRIAR COMPLEXIDADE DESNECESSÁRIA.

SEM REGRESSÃO.

==========
FIM DO COMANDO MESTRE
==========

```

Esse é o bloco que eu considero **a versão correta para cristalização**: ele incorpora o
```

conteúdo do bloco online — especialmente a parte que eu tinha deixado escapar sobre **Hermes como centro, `UNKNOWN != PASS`, reconciliação, watchdog/autocura, Gateway/Hub, Grok/xai-oauth, Tailscale/Remote Bridge, GitHub/CI/CD e o loop autônomo** — junto com todo o pacote de **Agent XP, HUD, runtime Casa/VPS, sandbox, Mermaid, provas e Deltas** construído nesta sessão. :contentReference[oaicite:1]{index=1}

E uma correção importante da minha própria conduta: **eu não vou mais compactar um bloco desses só para deixá-lo “bonito”.** Quando você pedir **cristalização**, a regra será **preservar conteúdo + acrescentar conteúdo + corrigir contradições**, não resumir.
```

[1]: https://pt.anotepad.com/notes/ym4ch5s5 "centraldark - Bloco de Notas Online" AGORA VAMO MANTER ESE TAMBEM IGUAL POREM ORGANIZADO CENTRALDARK → TLC23_OPEN → DESTINATION_LOCK → ACORDAR → CALIBRAR → docs/CONSCIENCIA/04-MCP-E-CONECTORES/tlc23.pdf REUSE → EXTEND_EXISTING → CREATE_MINIMAL REALITY_JUDGE → FIXED_POINT → TLC23_CLOSE → REGISTRA → HANDOFF → BOOT AUREOLA-BAHIA-BIBLICA.yml !CENTRALDARK → !tlc23 → DESTINATION_LOCK → #ACORDAR → ... → FIXED_POINT → !tlc23 → REGISTRA → HANDOFF → BOOT. MASTER-CYCLE.yml !CENTRALDARK → #ACORDAR → !CALIBRAR → TRIGGERS.yml !CENTRALDARK ACORDAR, CALIBRAR, MULTIBUSSOLA... scripts/trigger-fabric/centraldark_master_cycle.py CHAIN ACORDAR MASTER_INGRESS_ALIASES ENGINE_BINDING.yml master_cycle_binding.expands_to !tlc23 CREATE_NEW_GATE_SYSTEM = F EXTEND_EXISTING_MASTER = V !CENTRALDARK !CENTRALDARK → TLC23_OPEN → DESTINATION_LOCK → #ACORDAR → !CALIBRAR → !MULTIBUSSOLA → PEGATUDO → DÉBITO_COGNITIVO → READERS_MEASURE → META_LUPA → RENDEZVOUS → RECONCILE → OWNER → ONE_WRITER → PATCH/EXECUÇÃO → TESTE → CONTRAPROVA → REALITY_JUDGE → FIXED_POINT → TLC23_CLOSE → REGISTRA → HANDOFF → BOOT !tlc23 2X!CENTRALDARK =!CENTRALDARK UNIVERSAL-ENGINE-METHOD docs/CONSCIENCIA/ADMINISTRAVEIS.md ADMINISTRAVEIS.json ☒ COPIAR, ☒☒ EDITAR OFICIAL, admin-patterns-ui.js #ACORDAR, !CALIBRAR, !MULTIBUSSOLA #ACORDAR, !CALIBRAR, !MULTIBUSSOLA !CENTRALDARK docs/CONSCIENCIA/01-IDENTIDADE-E-
```

REGRAS/SOUL-SOL.md CENTRALDARK_AI_BOOT.yaml GitHub persistente + runtime medido SOURCE, SOUL_SHA, MAIN_HEAD, UPDATED_AT, STATUS, TLC23_GATE !CENTRALDARK main arquivo local mais novo → ganha autoridade automaticamente SYSTEM-PROMPT-ADMIN.md PROMPT-MESTRE.md HISTORICAL_REDIRECT !CENTRALDARK → TLC23 TLC23 já integrado ao executor mestre completo EXTEND_EXISTING !CENTRALDARK ARGET_ARCHITECTURE_DERIVED = TRUE

CREATE_PARALLEL_SYSTEM = FALSE

EXTEND_EXISTING_MASTER = TRUE

REUSE_EXISTING_ADMIN_PANEL = TRUE

DERIVED_SOUL_PERSONALIZATION = TRUE

IMPLEMENTATION = NOT_STARTED

→ capacidade_disponivel ∧ autoridade ∧ escopo_determinado → executar
→ ci_pending → não_pass
→ merge → não_deploy
→ runtime_sem_prova → unknown
→ prova_real → green

Sim. Você tem razão: pra desenhar certo eu precisava primeiro ler a fonte da verdade.

Eu já fiz a parte que faltava do PDF tlc23 e a prova correta vai por aqui:

O que o tlc23.pdf realmente traz

Ele fala de:

→ Fórmulas de Horn

→ Satisfazibilidade de cláusulas

→ Modelo mínimo de Herbrand

Ou seja: não é Bayesianismo.

É derivação lógica por regras.

Como eu vou provar os gates do PDF
1) Traduzir o PDF para gates operacionais

O núcleo do PDF vira isso:

FATOS

→ coisas já verdadeiras agora

REGRAS

→ se A e B, então C

CONTRADIÇÃO

→ se uma regra leva a FALSO, o estado quebra

FECHAMENTO / PONTO FIXO

→ continuar derivando até não surgir nada novo

2) Estrutura lógica do gate

Vou tratar cada gate assim:

entrada

→ fato medido

regra

→ cláusula de Horn

saída

→ conclusão derivada

checagem

→ deu contradição ou não?

3) Fórmula mental da Casa

Fica assim:

FATOS MEDIDOS

→ runtime

→ GitHub

→ Casa

→ VPS

→ MCP

→ Gateway

→ Hub

REGRAS DO CONTRATO

→ capacidade_disponivel ∧ autoridade ∧ escopo_determinado → executar

→ ci_pending → não_pass

→ merge → não_deploy

→ runtime_sem_prova → unknown

→ prova_real → green

DERIVAÇÃO

→ executa o encadeamento

SE DER FALSO / CONFLITO

→ gate vermelho

SE NÃO DER CONTRADIÇÃO E FECHAR

→ gate válido

4) Como isso entra no teu comando

Do jeito que você pediu:

!CENTRALDARK

→ carrega fatos atuais

→ aplica gates do tlc23.pdf

→ deriva ações válidas

→ bloqueia contradições

→ chega no ponto fixo

→ só então desenha / decide / executa

Em setas, seco

PDF tlc23

→ define forma da regra

estado atual medido

→ vira fatos

contrato da Casa

→ vira cláusulas de Horn

motor de derivação

→ aplica regras até parar

resultado

→

sem contradição = coerente
com contradição = quebrado
sem prova suficiente = unknown
O desenho certo dos gates

Vou usar esta lógica:

FATO

→ bolinha base

REGRA

→ seta de encadeamento

NÚCLEO

→ conclusão central derivada

CARACOL

→ camadas de derivação que se aproximam do núcleo

PONTO FIXO

→ quando o caracol para de gerar novidade

Então teu caracol não é só bonito.

Ele representa:

periferia

→ fatos dispersos

camadas internas

→ regras aplicadas

núcleo

→ consciência / índice / mapa do cérebro / brain / leia-primeiro

O que eu errei antes

Eu errei em:

→ não usar primeiro o PDF como base formal

→ tentar responder sem fechar a lógica do material-fonte

→ misturar uma ideia de probabilidade onde o PDF é de derivação lógica

Correção cristalizada:

tlc23.pdf = gate lógico estrutural

não = matemática bayesiana explícita

Mas dá sim pra chegar ao mesmo tipo de decisão prática por derivação lógica, que é o que você queria provar.

Resultado prático

Se eu for fazer isso direito agora, a sequência é:

ler fonte da verdade

→ tlc23.pdf

mapear gates

→ fatos / regras / contradições / ponto fixo

encaixar no !CENTRALDARK

→ como pré-gatilho lógico

só depois desenhar o caracol

→ com significado real, não decorativo

Resposta seca à tua pergunta

Como vai provar os gates do PDF?

Assim:

tlc23.pdf

→ extrair regras de Horn

→ transformar contrato da Casa em premissas e conclusões

→ rodar encadeamento lógico

→ detectar contradição ou ponto fixo

→ só validar como green quando a derivação bater com a realidade medida

Sim. Vou manter esse bloco como está conceitualmente, apenas organizado e cristalizado, sem misturar com o Delta 2 e sem substituir conteúdo já definido.

E o ponto que você acabou de fixar fica incorporado como camada metodológica TLC23:

!CENTRALDARK
→ TLC23_OPEN
→ DESTINATION_LOCK
→ #ACORDAR
→ !CALIBRAR
→ !MULTIBUSSOLA
→ PEGATUDO
→ DÉBITO_COGNITIVO

→ READERS_MEASURE

→ META_LUPA

→ RENDEZVOUS

→ RECONCILE

→ OWNER

→ ONE_WRITER

→ PATCH/EXECUÇÃO

→ TESTE

→ CONTRAPROVA

→ REALITY_JUDGE

→ FIXED_POINT

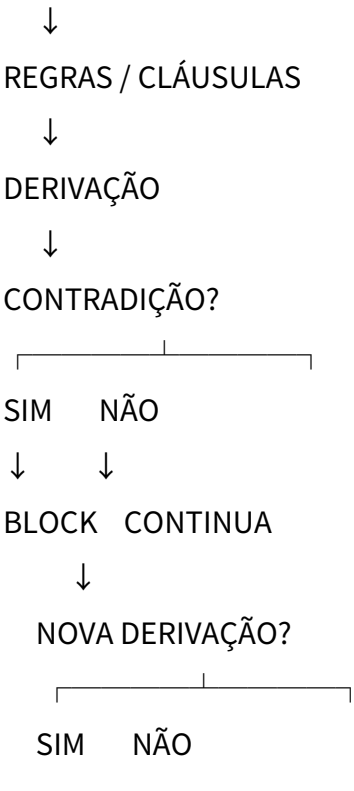
→ TLC23_CLOSE

→ REGISTRA

→ HANDOFF

→ BOOT

Camada formal TLC23



↓ ↓

CONTINUA FIXED_POINT

↓

REALITY_JUDGE

↓

PROVA REAL

↓

GREEN / UNKNOWN

E preservamos exatamente as regras já cristalizadas:

capacidade_disponivel ∧ autoridade ∧ escopo_determinado → executar

ci_pending → não_pass

merge → não_deploy

runtime_sem_prova → unknown

prova_real → green

Arquitetura de autoridade

!CENTRALDARK

↓

TLC23

↓

MASTER-CYCLE

↓

ACORDAR

↓

CALIBRAR

↓

MULTIBUSSOLA

↓

ONE_WRITER

↓
EXECUÇÃO

Com:

CREATE_NEW_GATE_SYSTEM = FALSE
EXTEND_EXISTING_MASTER = TRUE
REUSE_EXISTING_ADMIN_PANEL = TRUE
ARGET_ARCHITECTURE_DERIVED = TRUE
DERIVED_SOUL_PERSONALIZATION = TRUE

E a regra de fonte continua:

!CENTRALDARK
↓
TLC23
↓
SOURCE
↓
SOUL_SHA
↓
MAIN_HEAD
↓
UPDATED_AT
↓
STATUS
↓
TLC23_GATE

Sem criar um sistema paralelo. O tlc23 continua sendo uma camada/gate do executor mestre existente, não um novo executor.

Também fica preservado:

LOCAL MAIS NOVO
↓
GANHA AUTORIDADE AUTOMATICAMENTE

e:

GITHUB PERSISTENTE
+
RUNTIME MEDIDO
+
REALITY_JUDGE
+
PROVA REAL
E a regra operacional que fecha tudo
REUSE_EXISTING
↓
EXTEND_EXISTING
↓
CREATE_MINIMAL
↓
TEST
↓
CONTRAPROVA
↓
REALITY_JUDGE
↓
FIXED_POINT
↓

REGISTRA

↓

HANDOFF

↓

BOOT

Portanto, não vou reescrever isso como uma arquitetura nova. A partir daqui, esse material deve ser tratado como a estrutura consolidada existente, e novos deltas entram por extensão mínima sobre ela.